心音边缘 AI 学习课：L01-L17 离线读物

用途：把前 17 课的原理、真实数字和思考题装订成一份可以离线翻看的资料。
生成日期：2026-10-16
配套仓库：E:\UserData\Documents\ChatGPT\FYP
姊妹文档：learning/lessons_L18-L36.md

怎么用这份文档

每一课按同一个结构写：

1. **这一课要回答的问题** —— 一句话。
2. **原理** —— 为什么是这样，用生活里的例子讲。
3. **本仓库的真实数字** —— 全部来自 `learning/lessonXX_*.py` 的实际运行输出。
4. **思考题** —— 想完再看最后的答案部分。

这份文档里的每个数字都可以自己复现。命令的统一格式是：

```
.\.venv\Scripts\python.exe learning\lesson01_wav_info.py
```

把文件名换成对应的脚本即可。**脚本里不含任何随机数的地方，重跑数字完全一致**；含随机的地方（L05 到 L09）用了固定种子，同一台机器上也一致。

第一部分：软件基础（L01-L10）

这一段的目标是：**先把 Python、NumPy、PyTorch**
这三样工具用熟，用最简单的例子，不要一上来就碰心音模型。

L01 用 Python 读取 WAV 的基本信息

这一课要回答的问题

一个音频文件里，到底装了些什么？

原理

WAV 文件可以理解成两样东西：

- **一排数字**：每个数字代表某一瞬间的空气压力。
- **一张说明书**：写着怎么解释这排数字。

说明书里有四项必须知道：

项目	含义	类比
声道数	单声道还是立体声	一路录音还是两路
采样率	每秒记录多少个点	每秒拍多少张照片
样本宽度	每个点占几个字节	每张照片用多少种颜色
样本总数	一共记录了多少个点	总共拍了多少张

时长不用存在文件里，**算出来**：

```
时长 = 样本总数 / 采样率
```

本仓库的真实数字

```
文件：data/processed/synthetic-pcg/training_data/90001_AV.wav
channels           = 1
sample_rate        = 4000
sample_width_bytes = 2
frames             = 74438
duration_sec       = 18.61
duration_min       = 0.31
```

三个要记住的点：

1. **4000 Hz 是我们项目的 AI 输入采样率**（ADR-002 冻结）。
2. **2 字节 = 16 bit**，这是 WAV 最常见的整数格式。
3. `74438 / 4000 = 18.61` 秒——时长是算出来的，不是读出来的。

思考题

1. 如果采样率是 4000 Hz、时长 18.61 秒，样本总数应该是多少？
2. 为什么时长不直接存在文件头里？

L02 用 CSV 统计 split 和标签分布

这一课要回答的问题

数据集里有多少人、多少段录音，标签是怎么分布的？

原理

CSV 就是一张用逗号隔开的表格。Python 读进来之后，每一行变成一个"字典"：列名是键，单元格是值。

统计的核心动作是**去重计数**：

```
录音有 79 条，但受试者没有 79 个人——
因为同一个人可能录了好几个部位（AV / MV / PV / TV）。
```

类比：一个班有 40 个学生，但交上来 120 份作业。数作业份数和数人数是两回事。
这就是为什么划分训练集和测试集必须按"人"，不能按"作业"。

本仓库的真实数字

```
train subjects  = 16          <- 去重后的人数
total recordings = 79         <- 录音条数

split counts  = {train: 54, test: 14, val: 11}
label counts  = {0: 43, 1: 36}
split x label = {(train,0): 29, (train,1): 25, (test,0): 8,
                 (test,1): 6, (val,0): 6, (val,1): 5}
location counts = {MV: 22, PV: 20, TV: 19, AV: 18}
```

两个值得注意的地方：

1. **79 条录音来自 24 名受试者**，平均每人 3 条多。按录音数算和按人数算是完全不同的两个数。
2. 标签接近平衡（43 : 36），但这是子集。完整数据集是 **695 健康 : 179 异常**，比例 3.9:1，那个才是真实的不平衡情况。

思考题

1. 为什么"79 条录音"不能当成"79 个样本"来看待统计意义？
2. 如果按录音随机划分训练集和测试集，会出什么问题？

L03 NumPy 读取 PCM 并计算 RMS

这一课要回答的问题

怎么用几个数字，描述一段声音"有多响"？

原理

声音的原始数据是整数，有正有负。**平均值不行**：正负一抵消，结果接近 0，看不出响不响。

所以用 **RMS（均方根）**，三步：

```
1. 每个数平方        -> 负数变正数，大数被放大
2. 求平均
3. 开根号
```

写成公式：

```
RMS = sqrt( mean( x² ) )
```

> 类比：一群人报"偏离平均值多少"。你不用正负号互相抵消，而是把偏离量平方后取平均，再开根号。结果永远是个正数，代表"典型偏离有多大"。

为什么要先除以 32768？ 因为 16-bit 整数的范围是 -32768 到 32767。除完之后，所有值都落在 -1 到 1 之间，方便统一处理。

本仓库的真实数字

```
文件：data/raw/circor/subset/training_data/9979_MV.wav
shape = (84608,)      <- 84608 个采样点
dtype = int16
min   = -29835
max   = 32767
mean  = -14.24        <- 几乎是 0
rms   = 0.0815578
peak  = 0.9999695
```

三个要会读的地方：

1. **mean 只有 -14**，相对于 32767 的满量程可以忽略——说明没有明显的直流偏置，信号是"围绕 0 上下波动"的。
2. **max 正好是 32767**，这是 int16 的上限。**出现上限值意味着可能有削顶（clipping）**，那一段声音已经失真了。这是数据质量检查的一个信号。
3. **RMS 0.0816** 是归一化之后的响度，这个数字后面带通、切窗、特征提取时都会用到。

思考题

1. 为什么不直接用平均值衡量响度？
2. 如果发现 max 等于 32767，你会先怀疑什么？

L04 用 NumPy 切 2 秒窗口

这一课要回答的问题

一段 18 秒的录音，怎么变成模型能吃的 2 秒小片段？

原理

模型的输入长度是固定的（我们用的是 2 秒），所以要把长信号切成等长的小段。这里有两个参数：

参数	含义	类比
window（窗长）	每段多长	尺子有多长
hop（步长）	每次往前挪多远	每次挪几厘米

窗口数量这样算：

```
窗口数 =（总样本数 − 窗长）/ 步长 + 1      （只取整数部分）
```

当步长小于窗长时，相邻窗口会重叠。
这是故意的：增加样本量，也保证不会有特征正好卡在两个窗口的接缝上。

本仓库的真实数字

```
文件：data/raw/circor/subset/training_data/9979_MV.wav
sample_rate    = 4000 Hz
window_samples = 8000      <- 2.0 秒
hop_samples    = 4000      <- 1.0 秒
number of windows = 20

每个窗口的 RMS：
窗口 0  (起点      0) 0.07799
窗口 1  (起点   4000) 0.07030
窗口 2  (起点   8000) 0.06664
...
窗口 19 (起点  76000) 0.08580
```

三个观察：

1. **每 4000 个点切一次**，也就是每 1 秒切一次，所以窗口之间**重叠一半。**
2. **同一段录音里，各窗口的 RMS 在 0.067 到 0.088 之间波动**——说明响度不均匀。这正是需要"按窗口筛选质量"的原因（L19 会展开）。
3. 20 个窗口来自同一个人。**它们不是 20 个独立样本。** 这是 L16 患者级聚合的伏笔。

思考题

1. 如果步长改成 8000（等于窗长），窗口数会变成多少？重叠还存在吗？
2. 为什么窗口只覆盖到 76000，而不是 84608？

L05 PyTorch 最小训练循环

这一课要回答的问题

模型是怎么"学会"一条直线的?

原理

训练就是四步循环，重复很多遍：

```
1. 前向：拿当前参数算预测
2. 损失：预测和正确答案差多少
3. 反向：算出每个参数该往哪个方向改
4. 更新：按学习率挪一小步
```

> 类比：下山找最低点。你看不见整座山，但能感觉到脚下的坡度（梯度），于是每次朝下坡方向走一小步。**学习率就是步子的大小**：太大容易冲过头，太小走得太慢。

本仓库的真实数字

任务：拟合 `y = 2x + 1`，200 轮，学习率 0.01。

```
epoch   loss        weight    bias
0       1.585357    0.006167  0.545715
40      0.836576    0.482482  0.797525
80      0.468365    0.845009  0.909757
120     0.268088    1.120930  0.959779
160     0.154657    1.330935  0.982073
200     ---------   ---------  --------

final weight = 1.487283
final bias   = 0.991847
```

最关键的一点：它没有学会。

正确答案是 weight = 2.0、bias = 1.0。跑了 200 轮，weight 只到 **1.487**，还差得远。

原因是**学习率太小**（0.01），200 轮不够走到终点。对比 L10：同一个任务、同一个模型，只把学习率改成 0.1，结果就是 **weight = 1.999998、bias = 1.0。**

这说明什么？训练不收敛时，先怀疑学习率和轮数，不要急着换模型。

思考题

1. 为什么 weight 从 0.006 一路涨到 1.487，而不是直接跳到 2.0?
2. 如果学习率设成 10，会发生什么?

L06 打印一次完整训练步骤

这一课要回答的问题

一轮训练的内部，每个数字到底怎么来的?

原理

L05 把循环跑了 200 遍，看不清细节。这一课**只跑一轮**，而且只用 3 个样本，方便手算。

数据：

```
x = [-1, 0, 1]
y = 2x + 1 = [-1, 1, 3]
```

模型只有一个参数和一个偏置：预测 = weight × x + bias。

本仓库的真实数字

```
=== 训练前 ===
weight = -0.0074868        <- 随机初始化的结果
bias   =  0.5364436

prediction = [0.5439, 0.5364, 0.5290]
target     = [-1., 1., 3.]
loss       = 2.9015532     <- 预测离目标很远

weight.grad = -2.6766491   <- 负数
bias.grad   = -0.9271128   <- 负数

=== 更新一次后 ===
weight = 0.2601781         <- 变大了
bias   = 0.6291549         <- 也变大了
```

怎么读梯度?

梯度的意思是"如果我把这个参数调大一点，损失会怎么变"。

梯度是 **负数**，说明"把参数调大，损失会变小"。所以更新之后，weight 和 bias **都变大了**——这正是我们想要的方向。

> 类比：你摸到坡是往下走的，就朝那个方向迈一步。梯度就是坡的方向和陡峭程度。

思考题

1. 如果梯度是正数，参数会往哪个方向变?
2. 为什么这里的 loss 有 2.9 那么大?

L07 Dataset 和 DataLoader

这一课要回答的问题

样本变多以后，怎么一批一批喂给模型?

原理

两个角色分工：

角色	负责什么	类比
Dataset	回答"第 i 个样本是什么"	书架
DataLoader	决定"一次拿几本、按什么顺序拿"	每次去拿书的人

为什么要分批?

1. 内存装不下全部数据。
2. 一次只看一个样本，梯度噪声太大、训练慢；一次看全部，又太稳、容易卡在坏点上。**分批是折中。**

为什么要打乱顺序（shuffle）？ 如果样本按类别排好序，模型会先连续看到 10 个"健康"再连续看到 10 个"有病"，每批的方向都偏得很厉害。打乱之后每一批都接近整体分布。

本仓库的真实数字

```
样本数 20（10 个类别 0 聚在 (-1,-1) 附近，10 个类别 1 聚在 (1,1) 附近）
batch_size = 4        -> 每轮跑 5 批
shuffle    = True
训练 50 轮

epoch 10  loss 0.5566  准确率 1.0
epoch 20  loss 0.3010  准确率 1.0
epoch 30  loss 0.2050  准确率 1.0
epoch 40  loss 0.1502  准确率 1.0
epoch 50  loss 0.1146  准确率 1.0
```

注意一个现象：第 10 轮准确率就已经是 1.0 了，但 loss 还在从 0.56 降到 0.11。

这说明**准确率是个粗糙的指标**：只要概率跨过 0.5 就算对，哪怕只比 0.5 高一点点。loss 更细，它衡量"模型有多自信"。分类问题只看准确率，会漏掉很多信息。

思考题

1. 为什么准确率到 1.0 之后，loss 还能继续下降?
2. 如果不开 shuffle，训练会有什么变化?

L08 分类指标：准确率、混淆矩阵、精确率、召回率、F1

这一课要回答的问题

模型判对了几个不算数——那到底该看什么?

原理

假设 10 个人来检查：**4 个人真的有病（标签 1），6 个人健康（标签 0）。**

模型给每人打一个分数，分数越高越像有病。看结果不能只数"对了几个"，要分成四种情况：

情况	英文	含义	在筛查里的后果
TP	True Positive	真有病，判有病	正确
TN	True Negative	真健康，判健康	正确
FP	False Positive	健康，误判有病	虚惊一场、多余检查
FN	False Negative	有病，漏判健康	**最危险**，病人被漏掉

六个指标：

```
accuracy             = (TP + TN) / 总数              全判对了多少
precision            = TP / (TP + FP)               说是"有病"的人里有多少真有病
recall / sensitivity = TP / (TP + FN)               真有病的人被抓出来多少
specificity          = TN / (TN + FP)               真健康的人被正确排除多少
F1                   = 2 × P × R / (P + R)            精确率和召回率的平衡；忽略 TN
balanced accuracy    = (recall + specificity) / 2   正负两类的表现取平均
```

什么时候用哪个?

- 数据一边倒时，**accuracy 会骗人**（全猜多数类也有很高准确率）。
- 医学筛查**最怕 FN**，所以优先看 recall。

- 但 recall 不能无限拔高（全说有病就 100%），所以要配 precision 一起看。
- **balanced accuracy** 在类别不平衡时比 accuracy 可靠。

本仓库的真实数字

```
=== threshold = 0.5 ===
TP=3 TN=5 FP=1 FN=1
accuracy             = 0.8000
precision            = 0.7500
recall / sensitivity = 0.7500
specificity          = 0.8333
F1                   = 0.7500
balanced accuracy    = 0.7917

=== threshold = 0.3 ===
TP=4 TN=3 FP=3 FN=0
accuracy             = 0.7000
precision            = 0.5714
recall / sensitivity = 1.0000
specificity          = 0.5000
F1                   = 0.7273
balanced accuracy    = 0.7500
```

阈值一变，四个格子全变：

阈值	TP	TN	FP	FN	recall	precision	发生了什么
0.5	3	5	1	1	0.75	0.75	漏了 1 个病人
0.3	4	3	3	0	1.00	0.57	不漏病人，但误报变多

规律：

```
阈值降低 -> 更容易判"有病" -> recall 升，FP 变多，precision 降
阈值升高 -> 更保守         -> FP 变少，FN 变多，recall 降
```

注意 accuracy 反而从 0.80 掉到了 0.70，但它不该是决策依据：0.3 的阈值虽然少对了 1 个健康人，却多救回了 1 个病人。

思考题

1. 如果漏掉一个病人比误报严重得多，应该优先让哪个指标高？
2. 在 90% 都是健康人的数据里，为什么只看 accuracy 会骗人？

L09 训练集、验证集、测试集和过拟合

这一课要回答的问题

为什么"训练时表现很好"不能说明模型好？

原理

数据要分成三份，各干各的活：

数据	用来干什么	类比
训练集	学参数	平时做的练习题
验证集	调超参数、选阈值	模拟考
测试集	**只看一次**，出最终成绩	期末考

过拟合的意思是：模型把训练数据里的噪声也背下来了，所以训练分数很好看，换一批数据就崩。

> 类比：考试前把练习册的答案全背下来。练习册考 95 分，换一套没见过的卷子就完了。这不是"蒙"，是"背题"。

本仓库的真实数字

用多项式去拟合 30 个带噪声的点（15 训练 / 8 验证 / 7 测试）：

```
=== 1 次多项式（一条直线）===
train MSE =       0.6274
val   MSE =       1.3209
test  MSE =       2.7181

=== 3 次多项式 ===
train MSE =       0.5804
val   MSE =       3.5077
test  MSE =       9.5971

=== 9 次多项式 ===
train MSE =       0.2765        <- 训练误差最小
val   MSE =  254572096.6        <- 验证误差爆炸
test  MSE = 534454482416.2      <- 测试误差更爆炸
```

这是整份文档里最直观的一次演示。

9 次多项式的**训练**误差最小（0.28），看起来最厉害；但它的**验证和测试**误差大到没有意义（10 的 8 次方和 10 的 11 次方）。模型为了穿过每一个训练点，在点与点之间疯狂摆动，完全失去了预测能力。

三条结论：

1. **只看训练误差选模型，一定会选到最过拟合的那个。**
2. 模型复杂度要和数据量匹配。15 个点撑不起 9 次多项式。
3. **测试集只看一次。** 如果反复拿它调参，它就变成了验证集，最终成绩也就不再可信。

思考题

1. 为什么 9 次多项式的训练误差最小，却是最差的模型？
2. 如果验证集和测试集的结论冲突，应该信哪个？

L10 模型保存、加载和可复现性

这一课要回答的问题

训练好的模型怎么存下来，怎么保证下次跑出一样的结果？

原理

存两样东西，缺一不可：

存什么	为什么
模型权重（state_dict）	学到的数字
配置（JSON）	架构、超参数、数据来源、种子

> 类比：存一份乐谱的演奏录音（权重）之外，还要写下"用哪把乐器、什么速度"（配置）。只给录音，别人没法重演。

为什么只存权重，不存整个模型？ 因为 PyTorch
存整个对象时会把代码结构也序列化进去，换个版本就可能加载不了。**只存张量、用代码重建结构，更稳。**

可复现性靠三件事：固定随机种子、记录所有超参数、把配置和权重一起存。

本仓库的真实数字

任务还是 `y = 2x + 1`，但这次学习率用 0.1（对比 L05 的 0.01）：

```
final loss  = 0.0
weight      = 1.999998
bias        = 1.0

保存后重新加载，再算一遍：
max difference = 0.0            <- 完全一致
first 3 predictions = [-0.9999984, -0.9595944, -0.9191905]
```

三个要点：

1. **学习率 0.1 时，200 轮就精确收敛到 2.0 和 1.0**；而 L05 用 0.01 跑了 200 轮才到 1.487。**同一份代码，只改一个数字，结果天差地别。**
2. **保存前后输出差 0.0**，说明存取没有引入任何误差。
3. 产物落在 `learning/outputs/lesson10/`：`linear_state.pt`、`config.json`、`metrics.json`。**三个文件是一套**，缺了配置就无法复现。

思考题

1. 为什么"只存权重不存配置"会让实验无法复现？
2. 对比 L05 和 L10，学习率从 0.01 改成 0.1 带来了什么变化？

第二部分：心音 AI 核心起步（L11-L17）

这一段开始接触真实的心音数据。核心线索是：**把一段波形，一步步变成模型能吃的数字，再学会评价它。**

L11 FFT 和频谱

这一课要回答的问题

一段声音里，到底有哪几种频率？各占多少？

原理

波形图告诉你"某一刻振幅多大"，但它没告诉你"声音是由哪些频率组成的"。

FFT 干的就是这件事：**把随时间变化的波形，翻译成"每种频率各有多少能量"。**

> 类比：棱镜把一束白光分成七彩。FFT 把一段混合的声音，分成一条频谱。

三个必须知道的换算：

```
频率分辨率 = 采样率 / 点数
最高能看到的频率 = 采样率 / 2        （奈奎斯特频率）
```

为什么要加窗？ 因为信号被切成 1 秒，两端是"硬生生截断"的。硬截断会凭空产生高频成分（听起来像"啪"的一声），所以先让两端慢慢淡出，再算 FFT。

本仓库的真实数字

第一部分：合成信号（4000 Hz、1 秒，人为放进去三个频率）

```
synthetic signal top 3 frequencies:
    50.00 Hz  magnitude=1200.0000
   120.00 Hz  magnitude=600.0000
   400.00 Hz  magnitude=200.0000
```

三个频率一个不漏地找出来了，而且**幅度比 1200 : 600 : 200 正好等于 0.6 : 0.3 : 0.1**——和构造信号时给的振幅比例完全一致。这说明 FFT 是可信的。

第二部分：真实心音（9979_AV 的第一秒，只看 20~1000 Hz）

```
heart sound top frequencies in 20-1000 Hz:
    68.00 Hz  magnitude=40.1961
    64.00 Hz  magnitude=38.6886
    69.00 Hz  magnitude=33.1736
    67.00 Hz  magnitude=32.5283
    84.00 Hz  magnitude=31.5166
```

关键观察：真实心音的前五名全部挤在 64~84 Hz 这一小段里。

这可能出乎意料——心音不是应该分布在很宽的频段上吗？原因是：**能量最大的成分是低频的 S1/S2 主体**，而很多细节（比如高频杂音）幅度小得多，在"前五名"里根本排不上。这就是为什么后面还要做 mel 分带（L14），不能只看 Top 5。

思考题

1. 采样率 4000 Hz 时，频谱最高能看到多少 Hz？频率分辨率是多少（1 秒数据）？
2. 为什么真实心音的 Top 5 都挤在低频？

L12 FIR 带通滤波器

这一课要回答的问题

怎么只保留想要的频率范围，把其余的削掉？

原理

FIR 滤波器就是**一张配方卡**：一串数字，叫 taps。滤波就是拿着这张卡在信号上滑动，每个位置按配方把附近的采样点加权相加。

配方怎么来？三步：

```
1. 低通的配方是 sinc 函数（理想低通）
2. 带通 = 低通(上限) - 低通(下限)
3. 乘窗函数压振铃，再归一化到通带增益 1.0
```

> 类比：先做一把"1000 Hz 以下全通过"的筛子，再做一把"20 Hz 以下全通过"的筛子，前者减后者，中间那段 20~1000 Hz 就留下了。

三个必须分清的概念：

名词	含义
通带	想保留的频率范围
阻带	想扔掉的频率范围
过渡带	通带和阻带之间的斜坡，不可能垂直掉下去

本仓库的真实数字

为了看得清楚，先设计一个 20~200 Hz 的带通（真实管线是 20~1000 Hz）：

```
number of taps = 257
causal delay   = 128 samples = 32.0 ms（只影响实时路径）

filter gain at test frequencies:
    10.0 Hz  gain = 0.186
    20.0 Hz  gain = 0.498   <- 注意这个 0.5
    50.0 Hz  gain = 1.000
   120.0 Hz  gain = 0.999
   200.0 Hz  gain = 0.499   <- 也是 0.5
   400.0 Hz  gain = 0.001
   800.0 Hz  gain = 0.000
```

用一个三音信号验证（50 Hz 幅度 0.6、120 Hz 幅度 0.3、400 Hz 幅度 0.1）：

```
  freq     before      after   change
    50 Hz   599.850    599.339        1.0x
   120 Hz   299.925    299.122        1.0x
   400 Hz    99.975      0.000  1218700.0x
```

三个反直觉的地方：

1. **20 Hz 和 200 Hz 的增益是 0.5，不是 1.0。** 因为你填进函数里的截止频率，就是**过渡带的中心，也就是 -6 dB 点。**真正的通带比 20~200 更窄。
2. **400 Hz 被削掉了 100 万倍。** 这就是阻带的作用。
3. **实时滤波会晚 128 个样本（32 毫秒）。** 离线分析用 `filtfilt`（正着滤一遍、倒着再滤一遍）能把这段延迟抵消，但代价是必须拿到整段信号，不能流式处理。

思考题

1. 为什么 20 Hz 处的增益是 0.5 而不是 1.0?
2. 离线用 `filtfilt`、实时用单次卷积，各自付出什么代价?

L13 仓库数据流和心音基础：S1/S2、4 kHz、2 秒窗

这一课要回答的问题

从探头到模型，一共有多少道工序？心音的"咚、哒"到底是什么?

原理

把心脏想成一个有四扇门的泵：

- 收缩一开始，两扇**进门**的阀门（二尖瓣、三尖瓣）关上——这是**咚**，也就是 **S1**。
- 血被泵出去，收缩结束，两扇**出门**的阀门（主动脉瓣、肺动脉瓣）关上——这是**哒**，也就是 **S2**。

标签	含义
1 = S1	第一心音，房室瓣关闭
2 = systole	收缩期，S1 到 S2 之间；**杂音最常出现在这里**
3 = S2	第二心音，半月瓣关闭
4 = diastole	舒张期，S2 到下一次 S1
0 = unannotated	未标注或低质量，不能当训练窗

整条链：

```
探头 -> 模拟抗混叠 -> ADC 8 kHz -> 20 ms 分包
     -> 无线链路 -> 主机解包 -> 带限抽取到 4 kHz
     -> 20~1000 Hz 带通 -> 切 2 秒窗 -> 特征 -> 模型 -> 结果
```

为什么是 2 秒？ 因为 2 秒能装下约 3 个心动周期，至少看到 2 到 3 次完整的"咚-哒"。

本仓库的真实数字

以 `9979_AV` 为例：

```
sample rate  = 4000 Hz    单声道，16-bit PCM
duration     = 18.160 s
S1 count = 8   S2 count = 7
mean RR      = 0.6676 s
heart rate   = 89.9 bpm       (= 60 / 0.6676)
S1->S2 gap   = 0.3022 s       (收缩期宽度)

overall annotated part = 0.266     (18.16 秒里只有 26.6% 被标注)
2 秒窗总数（步长 1 秒）  = 17
标注比例 >= 0.5 的窗口   = 5
```

四个要点：

1. **89.9 bpm 是从 S1 的间隔算出来的**，不是从文件里读的。心率可以用 `60 / RR` 反推。
2. **RR 是 0.6676 秒**，所以 2 秒窗大约装 3 个心动周期。
3. **只有 26.6% 被标注。** CirCor 只标注高质量片段，剩下的 0 不是"正常"，而是"不知道"。
4. **17 个窗口里只有 5 个被保留**（要求标注比例 ≥ 0.5）。这是 L19 窗口筛选的前置知识。

思考题

1. 心率 150 bpm 时，一个 2 秒窗里大约有几个心跳？
2. 为什么不能按窗口随机划分训练集和测试集？

L14 log-mel 特征提取

这一课要回答的问题

8000 个数字太长了，怎么压成模型能吃的几十个数字？

原理

四步压缩：

```
1. 分帧：把 2 秒切成 78 个小格（每格 64 ms），逐格分析
2. 每格做 FFT：得到 129 个频率点
3. 归并成 40 个 mel 频带：mel 是按"人耳感觉"刻的尺子
4. 取 log 并做统计：压成 128 个数字
```

类比：

- **分帧** = 看电影要一帧一帧看，不是盯着整卷胶片。
- **FFT** = 把一杯混合果汁尝出里面有哪些水果。
- **mel** = 给耳朵做的尺子。人对低音变化敏感，对高音不敏感，所以高频刻度粗一些。
- **取 log** = 像地震震级。能量差了 100 万倍，直接用倍数没法比，用对数就变成了 1 到 6。

128 个数字是怎么来的：

```
40 个频带 × 3 项（平均、波动、90 分位）= 120
谱形状 5 项（重心、带宽、滚降点…）       =   5
时间统计 3 项（RMS、过零率、峰值因子）   =   3
合计                                      = 128
```

本仓库的真实数字

```
log-mel shape = (78, 40)      78 个时间格 × 40 个频带
value range   = -3.67 到 2.53 （log10 功率）
dynamic range = 6.20 个数量级

mel band widths = 30.8 Hz（第 0 带）到 69.8 Hz（第 39 带）
线性 40 带对照   = 每带 24.5 Hz

feature vector = (128,)       = 128 个数字
```

三个容易被误解的地方：

1. **mel 谱图是 78×40，但模型吃的是 128 个数，不是 40 个数。** 两者不要混。
2. **mel 在 20~1000 Hz 内只比线性分带"好" 2.3 倍。** 它的优势要到跨好几个八度（比如语音的 20 Hz 到 8 kHz）才明显。在这个项目里，它的作用比想象中温和。
3. **训练和部署必须用同一套参数**（`n_fft`、`hop`、`n_mels`、`fmin`、`fmax`）。改动任何一个都等于换了输入空间，模型直接失效。

思考题

1. 为什么格子从 64 ms 改成 128 ms 时，会更难看清"什么时候发生"？
2. 为什么要取 log？

L15 跑逻辑回归基线

这一课要回答的问题

最简单的模型能做到什么程度？

原理

逻辑回归只做两件事：

```
1. 加权求和：总分 = w1×特征1 + ... + w128×特征128 + b
2. 压成概率：概率 = 1 / (1 + e^(-总分))
```

学到的就是 **128 个权重加 1 个偏置，一共 129 个数字。**

> 类比：老师改卷子，每道题有分值，最后算总分。逻辑回归就是自己学出"每道题值多少分"的那个老师。

为什么训练前要标准化？ 因为 128 个特征的量纲不同——有的在 -1 到 1 之间，有的能到 900 多。不统一尺度的话，数值大的特征会天然占优势。

> 类比：一场考试语文满分 100、数学满分 10000，那总分最后只看数学。不是数学更重要，是它数字大。

本仓库的真实数字

小数据集（24 名受试者）：

```
windows = 943     features = 128
验证集选出的阈值 = 0.4213

window_level test：n=199  accuracy 0.568  BA 0.550  AUROC 0.568
patient_level test：n=4   accuracy 0.750  sensitivity 0.500  specificity 1.000
```

完整数据集（874 名受试者，5 折交叉验证）：

```
窗口选择 = all        患者级 AUROC = 0.775 ± 0.029
窗口选择 = annotated  患者级 AUROC = 0.789 ± 0.031
```

最重要的对比：

数据集	患者数	患者级 AUROC	该不该信
小数据集测试集	4	0.750	**不能信**，置信区间 0.0 到 1.0
完整数据集 5 折	874	0.775 ± 0.029	可信，达标（门槛 0.70）

同一个 0.75 左右，一个毫无意义，一个相当可信——区别只在样本量。

思考题

1. 为什么训练集准确率 93% 和测试集 AUROC 0.568 可以同时出现？
2. 模型一共学了多少个数字？

L16 患者级聚合和 AUROC

这一课要回答的问题

指标该算在"片段"上，还是算在"人"上？

原理

先看一个事实：9979_AV 这一条录音切出 20 个窗口，它们共享同一颗心脏、同一个杂音、同一个听诊位置。**它们不是 20 个独立样本。**

所以正确做法是：

```
患者分数 = 这个患者所有窗口概率的平均
```

类比：评一个班，不能把 4 个学生的 200
道题倒在一起算平均分——那样做题多的人占更大权重。正确做法是先算每个人的平均分，再横向比较。

为什么用 AUROC?

因为敏感度、特异度都必须先划一条阈值线，而 AUROC 不需要：

```
AUROC = 随机抽一个病人 + 随机抽一个健康人，
        病人分数比健康人高的概率

0.5 = 完全瞎猜          1.0 = 完美排序
```

本仓库的真实数字

```
window-level  test  (n=199)   AUROC 0.568
patient-level test  (n=4)     AUROC 0.750   (95% CI 0.0 到 1.0)
patient-level 完整 5 折 (n=874) AUROC 0.775 ± 0.029
```

从 0.568 跳到 0.750，不是模型变好了。 原因有两个：

1. 聚合成患者级后，窗口的随机噪声被平均掉，真实信号浮出来。
2. 样本量从 199 掉到 4，不确定性同时暴增。

一个具体的聚合例子：

```
患者 A：17 个窗口，概率全是 0.6        -> 患者分数 0.60
患者 B：2 个窗口，概率是 0.9 和 0.2    -> 患者分数 0.55
```

患者 B 有一个比 A 高得多的窗口，但平均之后反而更低。**这就是"多窗口患者会被平均掉"的实际效果。**

思考题

1. 为什么窗口级和患者级的指标不能互相比较?
2. 4 个人的 AUROC 0.750 和 874 个人的 0.775，你更信哪个?

L17 1D CNN 结构

这一课要回答的问题

CNN 和逻辑回归有什么本质不同?

原理

根本区别在输入：

模型	输入	类比
逻辑回归	128 个统计摘要	只读体检报告
1D CNN	8000 点**原始波形**	亲自听一遍

摘要把"症状什么时候出现"这个信息丢掉了，而 CNN 直接在波形上找形状。

CNN 的三个核心零件：

```
卷积核 = 一把小放大镜，在波形上滑动找局部形状
通道   = 同时用几把放大镜（1 -> 16 -> 32 -> 64 -> 64）
池化   = 每两个取最大，把长度砍半，保留最显著的
```

> 类比：卷积核是拿一把带刻度的尺子贴着波形一格一格滑过去，每滑一下问："这 15 个点组成的形状，像不像我要找的东西？"

本仓库的真实数字

```
参数量              = 41,105
单窗口 MACs         = 5,722,944      （约 570 万次乘加）
输入                = 8000 点原始波形（float16）

长度变化：8000 -> 1000 -> 250 -> 62 -> 15 -> 1（全局平均）

FP32 权重           = 162.0 KiB
INT8 权重估算       = 40.1 KiB
PC PyTorch 延迟     = 平均 0.75 ~ 1.04 ms（随机器变化）
```

三个要点：

1. **参数少不等于计算少。** 41,105 个参数很小，但一次推理要 570 万次乘加——因为卷积核要在 8000 点的波形上来回滑动。
2. **最后一步"全局平均池化"让输入长度可以变。** 不管进来 8000 点还是 4000 点，出口永远是 64 个通道，所以尾巴不用改。**但换窗长必须重新训练。**
3. **两条基线的完整对比：**

模型	窗口选择	患者级 AUROC（5 折）
逻辑回归	all	0.775 ± 0.029
逻辑回归	annotated	0.789 ± 0.031
1D CNN	all	0.811 ± 0.052
1D CNN	annotated	**0.834 ± 0.061**

CNN 平均更好，但**标准差也大一倍**（0.061 对 0.031），说明它上限更高、但更"看运气"。而且只有 5 折，这个差距还不能算"已被证明"。

思考题

1. 为什么这个模型敢直接吃 8000 点原始波形?
2. 为什么 41,105 个参数却是 570 万次运算?

思考题参考答案

L01

1. **样本总数** = 4000 × 18.61 ≈ 74438（读数也确实是 74438）。
2. **时长不用存**：它由样本总数和采样率唯一决定，存进去反而会出现两者不一致的隐患。存原始数据，其余都算出来。

L02

1. **79 条录音不独立**：它们来自 24 个人，同一个人不同部位的录音高度相关。当独立样本用，统计置信区间会被严重低估。
2. **按录音随机划分会泄漏**：同一个人的录音会同时出现在训练集和测试集里，模型等于"见过这个人"，测试分数虚高。必须按患者划分。

L03

1. **平均值不行**：声音是围绕 0 上下波动的，正负相消，平均值接近 0，看不出响度。RMS 先平方再平均再开根号，避开了这个问题。
2. **max = 32767 要怀疑削顶**：这是 int16 上限，说明那一段可能已经失真（被截平了）。先查这一段是不是录音增益太高。

L04

1. **步长改成 8000**：窗口数 = (84608-8000)/8000 + 1 = 10 个，窗口之间**完全不重叠**（首尾相接）。
2. **只覆盖到 76000**：最后一个完整窗口从 76000 开始，到 84000 结束。剩下的 608 个点不够一个完整窗口，所以丢掉。（也可以选择补零，但那样会引入假信号。）

L05

1. **一步一步涨**：每次更新只挪"学习率 × 梯度"那么小的一步。学习率 0.01 时步子很小，所以要很多轮才能靠近 2.0，200 轮还没到。
2. **学习率 10 会震荡甚至发散**：步子太大，会在最低点两侧来回跳，甚至越跳越远，损失不降反升。

L06

1. **梯度是正数**：说明"参数调大，损失也变大"，那就该往**减小**的方向调。
2. **loss 2.9 很大是因为预测几乎全是 0.53，而目标是 -1、1、3**：差得远，平方后自然大。初始权重是随机给的，还没学任何东西。

L07

1. **准确率饱和后 loss 还能降**：准确率只看"有没有跨过 0.5"，而 loss 看"模型的概率离正确答案有多远"。从 0.51 升到 0.99，准确率不变，但 loss 会大幅下降。
2. **不打乱顺序**：样本是按类别排好的，每一批要么全是 0 要么全是 1，梯度方向来回摆，训练不稳、收敛慢。

L08

1. **优先让 recall 高**：漏掉病人（FN）的代价最大。
2. **只看 accuracy 会骗人**：90% 健康时，全猜"健康"就有 90% 准确率，但一个病人都没找出来。要看 balanced accuracy、recall、AUROC。

L09

1. **9 次多项式训练误差最小却最差**：它为了穿过每一个训练点，在点之间剧烈摆动。训练误差小是"背下来了"，验证和测试误差爆炸才是它的真实能力。
2. **信测试集**：验证集是用来调参的，你在它身上试了很多次，会有信息泄漏。测试集只看一次，它才代表真实的泛化能力。（如果两者都很差，说明模型整体不行；如果验证好测试差，通常是调参调过头了。）

L10

1. **只存权重无法复现**：因为不知道模型结构、学习率、轮数、数据来源，光有权重也重建不出同样的实验。
2. **学习率 0.01 到 0.1**：同样是 200 轮，一个只到 weight 1.487，一个精确到 1.999998。**超参数的一个数字，决定实验成败。**

L11

1. **最高频率 2000 Hz**（4000/2），**分辨率 1 Hz**（采样率 ÷ 1 秒的点数 = 4000/4000）。
2. **Top 5 挤在低频**：因为能量最大的成分是 S1/S2 的主体，集中在低频。高频成分幅度小得多，排名上不去——这正是需要 mel 分带而不是只看 Top 5 的原因。

L12

1. **20 Hz 处增益 0.5**：因为填进去的截止频率就是过渡带的中心，也就是 -6 dB 点。真正的通带比 20~200 更窄。
2. **离线** `filtfilt`：零相位、输出对齐，但必须拿到整段信号，不能流式，边缘还要 padding。**实时单次卷积**：可以边来边算，但固定延迟 32 ms（257 taps 时）。

L13

1. **150 bpm 时约 5 个心跳**：RR = 60/150 = 0.4 秒，2 秒 ÷ 0.4 = 5 个周期。心率越高，2 秒里装的心跳越多。
2. **不能按窗口随机划分**：同一个患者的窗口高度相关，随机划分会让同一个人同时进训练集和测试集，测试分数虚高。

L14

1. **格子变长更难看清"什么时候发生"**：128 ms 的格子比 S1/S2 本身还长，两个音会糊在一起。频率倒是更清楚了——这就是时间与频率不可兼得的权衡。
2. **取 log 是因为动态范围太大**：实测功率谱跨 6.2 个数量级，直接比较会被最大能量主导；取对数后压成 6 左右的范围。

L15

1. **训练 93% 和测试 0.568 可以并存**：训练集分数高只说明模型"背下来了"（过拟合），测试集才是真实泛化能力。另外窗口标签是从患者标签复制来的弱标签，训练信号本身就脏。
2. **一共 129 个数字**：128 个权重 + 1 个偏置。

L16

1. **不能互相比较**：窗口级回答"这个 2 秒片段判得准不准"，患者级回答"这个人整体判得准不准"。而且窗口级把同一个人重复算了很多遍，独立性不成立。
2. **更信 874 人的 0.775**：4 个人的 95% 置信区间是 0.0 到 1.0，等于什么都没说；874 人的是 0.727 到 0.808，窄得多也稳定得多。

L17

1. **敢直接吃 8000 点**：因为第一层卷积核很宽（15 个采样点）配合 stride，一开始就把长度大幅压缩，而且模型内部自带标准化。它不需要人工做摘要——这正是 CNN 的价值。
2. **41,105 个参数却要 570 万次运算**：参数是"要记多少个数"，MACs 是"要算多少次"。卷积核虽小，但要在很长的波形上反复滑动，每个位置都要算一遍。

现在该做什么

这份文档覆盖的是**已经完成的 L01 到 L17**。它们都有对应的脚本，可以随时重跑：

```
.\.venv\Scripts\python.exe learning\lesson01_wav_info.py
.\.venv\Scripts\python.exe learning\lesson08_metrics.py
.\.venv\Scripts\python.exe learning\lesson12_fir_bandpass.py
```

下一步在姊妹文档里：`learning/lessons_L18-L36.md`（L18 到 L36）。

三份资料的定位：

文档	覆盖	状态
本文档	L01-L17	讲义 + 脚本都已完成，可重跑
`lessons_L18-L36.md`	L18-L36	讲义已读；L18-L24 待实跑，L25 起待开发板

按项目纪律记住一句话：证据不足时，只能写"候选"或"待实测"。